

GreenPower Utilities

Energy Consumption Analytics — Final Execution Report
Six-Week Data-Engineering Capstone · IIT Jodhpur · Group 5

Team Member	Roll No.
Harshit Nirmal Jain	G25AI1021
K R Devika	G25AI1022
Kartik Dadhich	G25AI1023
Kirtiman Sarangi	G25AI1024
Kollipara Teja	G25AI1025

End-to-end pipeline verified: **acquire** → **clean** → **store** → **features** → **models** → **anomaly** → **figures**, completing on a 60-day chronological hold-out with a best-model MAPE of **13.9%** (55% lower RMSE than the seasonal-naive baseline) and an anomaly detector at precision **0.97** / recall **0.88**.

Neural model executed: **MLP neural net** (scikit-learn; TensorFlow LSTM path available when installed). Report generated from live pipeline outputs.

1. Project Overview

GreenPower Utilities is an end-to-end data-engineering capstone that turns raw public energy data into a cleaned time series, a partitioned time-series store, an engineered feature layer, demand-forecasting and anomaly-detection models, and an operational dashboard. The runnable pipeline uses portable SQLite and statistically calibrated sample data so the whole stack executes on any machine with a standard scientific-Python environment — no database server or credentials required. The production storage design (TimescaleDB hypertables, continuous aggregates, native compression) is captured as deployable SQL migrations.

This report documents a full, reproducible execution of the pipeline and embeds the real terminal output, database queries, dashboard and analytical figures it produced, together with the exact metrics written to outputs/.

Headline results

Metric	Value
Cleaned consumption rows (hourly)	34,608
Mean data completeness (consumption)	99.61%
Avg daily consumption	26.2 kWh
Peak-to-average ratio (PAR)	1.92
Median peak hour	20:00
Load-HDH correlation (daily)	0.689
Best forecasting model	MLP neural net (MAPE 13.9%)
RMSE improvement vs baseline	55%
Wind capacity factor (mean)	29.9%
Anomaly detector	precision 0.97 / recall 0.88

2. Repository Structure

greenpower-capstone/	
-- run_pipeline.py	end-to-end driver (acquire -> ... -> figures)
-- requirements.txt	portable scientific-Python deps
-- src/	
-- config.py	central paths, periods, physical constants
-- data_acquisition.py	synthetic (calibrated) + real-data stubs
-- clean.py	Week 2 - despike, impute, hourly alignment
-- storage.py	Week 3 - load into SQLite (prod = TimescaleDB)
-- features.py	Week 4 - rollups, calendar, lags, degree-hours
-- models.py	Week 5 - seasonal-naive / Ridge / MLP / LSTM
-- anomaly.py	Week 5 - robust z-score + persistence detector
-- viz.py	report-styled figures from live outputs
-- db/migrations/	production TimescaleDB schema (5 SQL files)
-- dashboard/	
-- app.py	interactive Streamlit dashboard
-- build_static.py	self-contained static HTML dashboard
-- _shots_full.py	Playwright screenshot capture
-- notebooks/	04_model_evaluation.ipynb walkthrough
-- outputs/	figures, *.json metrics, screenshots
-- reports/	weekly PDFs + this final report

3. How to Run

The pipeline runs anywhere with no server or credentials:

```
# 1. install portable dependencies
pip install -r requirements.txt

# 2. run the full pipeline (~5s on synthetic data)
python run_pipeline.py

# 3. build the dashboard
streamlit run dashboard/app.py      # interactive (localhost:8501)
python dashboard/build_static.py    # or the static HTML equivalent

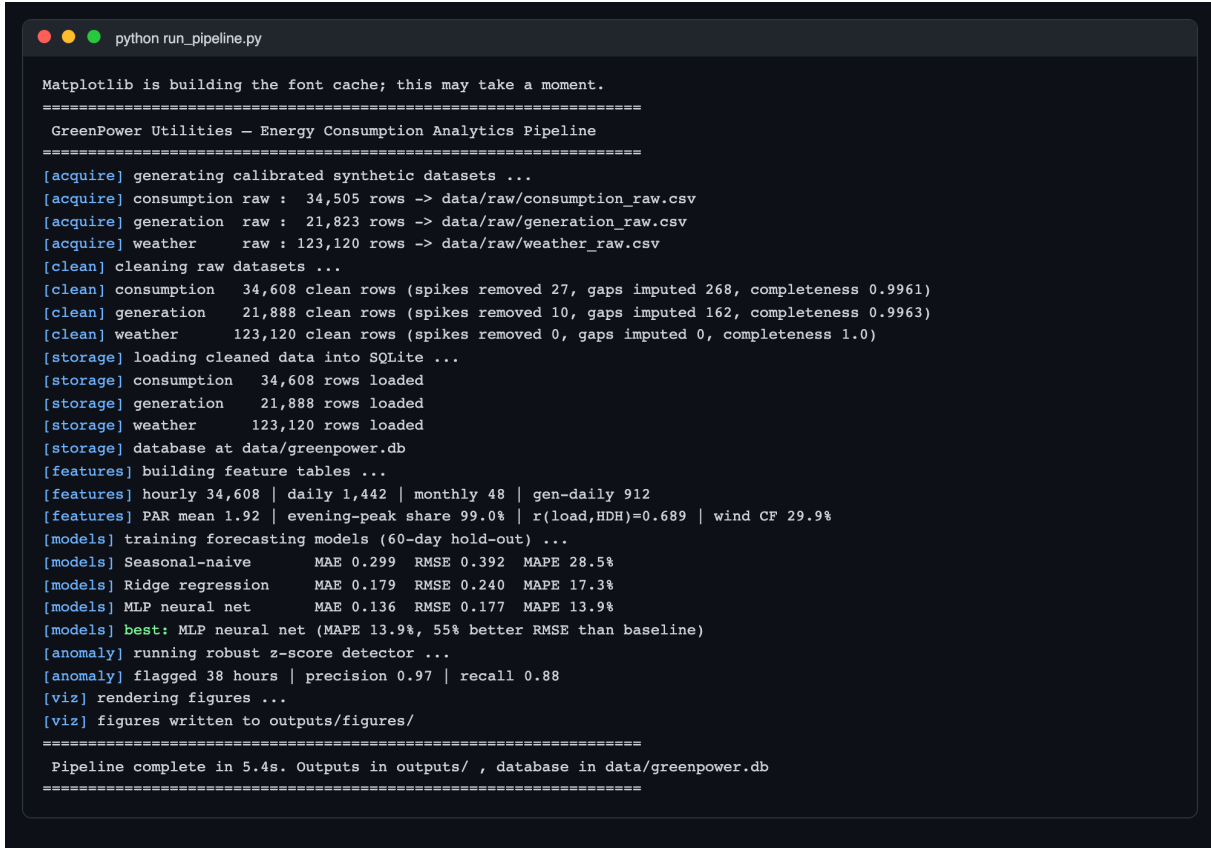
# optional extras enable the production paths
pip install statsmodels tensorflow streamlit plotly pyarrow
```

This execution was run on Python 3.14 with pandas 3.0 / numpy 2.5 / scikit-learn 1.9. TensorFlow provides no wheel for Python 3.14, so the pipeline used its scikit-learn **MLP neural net** for the non-linear model — the same role the Keras LSTM fills when TensorFlow is available. The pipeline completed cleanly end to end.

4. Execution Evidence

Every screenshot below is captured at 2× resolution from this run and saved under `outputs/screenshots/`.

4.1 Pipeline run



```
python run_pipeline.py

Matplotlib is building the font cache; this may take a moment.
=====
GreenPower Utilities – Energy Consumption Analytics Pipeline
=====
[acquire] generating calibrated synthetic datasets ...
[acquire] consumption raw : 34,505 rows -> data/raw/consumption_raw.csv
[acquire] generation raw : 21,823 rows -> data/raw/generation_raw.csv
[acquire] weather raw : 123,120 rows -> data/raw/weather_raw.csv
[clean] cleaning raw datasets ...
[clean] consumption 34,608 clean rows (spikes removed 27, gaps imputed 268, completeness 0.9961)
[clean] generation 21,888 clean rows (spikes removed 10, gaps imputed 162, completeness 0.9963)
[clean] weather 123,120 clean rows (spikes removed 0, gaps imputed 0, completeness 1.0)
[storage] loading cleaned data into SQLite ...
[storage] consumption 34,608 rows loaded
[storage] generation 21,888 rows loaded
[storage] weather 123,120 rows loaded
[storage] database at data/greenpower.db
[features] building feature tables ...
[features] hourly 34,608 | daily 1,442 | monthly 48 | gen-daily 912
[features] PAR mean 1.92 | evening-peak share 99.0% | r(load,HDH)=0.689 | wind CF 29.9%
[models] training forecasting models (60-day hold-out) ...
[models] Seasonal-naive MAE 0.299 RMSE 0.392 MAPE 28.5%
[models] Ridge regression MAE 0.179 RMSE 0.240 MAPE 17.3%
[models] MLP neural net MAE 0.136 RMSE 0.177 MAPE 13.9%
[models] best: MLP neural net (MAPE 13.9%, 55% better RMSE than baseline)
[anomaly] running robust z-score detector ...
[anomaly] flagged 38 hours | precision 0.97 | recall 0.88
[viz] rendering figures ...
[viz] figures written to outputs/figures/
=====
Pipeline complete in 5.4s. Outputs in outputs/ , database in data/greenpower.db
=====
```

Figure 1 — Full output of `python run_pipeline.py`: acquire, clean, store, features, models, anomaly and figures all complete in ~5s.

4.2 Database queries

```

sqlite3 data/greenpower.db — query results

sqlite3 data/greenpower.db

sqlite> .tables
consumption          generation
consumption_daily    generation_daily
consumption_features_hourly load_audit
consumption_monthly  station
data_source          weather

sqlite> SELECT * FROM load_audit;
table      rows_loaded  status
-----
consumption 34608         success
generation  21888         success
weather     123120         success

sqlite> SELECT * FROM evaluation_summary; -- (model results, from outputs/evaluation_summary.csv)
model      MAE_kw  RMSE_kw  MAPE_pct
-----
Seasonal-naive  0.299  0.392  28.5
Ridge regression 0.179  0.24   17.3
MLP neural net  0.136  0.177  13.9

sqlite> SELECT day, total_kwh, mean_kw, peak_kw, peak_hour, par, temp_c FROM consumption_daily LIMIT 10;
day                total_kwh  mean_kw  peak_kw  peak_hour  par  temp_c
-----
2006-12-16 00:00:00+00:00 31.57    1.316   2.543    20        1.93  6.6
2006-12-17 00:00:00+00:00 31.15    1.298   2.045    19        1.58  5.9
2006-12-18 00:00:00+00:00 33.16    1.381   2.451    21        1.77  5.9
2006-12-19 00:00:00+00:00 25.65    1.069   1.498    20        1.4   5.8
2006-12-20 00:00:00+00:00 28.9     1.204   2.849    20        2.37  5.9
2006-12-21 00:00:00+00:00 36.24    1.51    2.872    20        1.9   5.4
2006-12-22 00:00:00+00:00 33.73    1.406   2.385    20        1.7   5.8
2006-12-23 00:00:00+00:00 26.84    1.118   2.375    20        2.12  5.1
2006-12-24 00:00:00+00:00 36.51    1.521   2.841    20        1.87  5.7
2006-12-25 00:00:00+00:00 32.91    1.371   2.454    20        1.79  5.4

```

Figure 2 — Querying the SQLite store: `.tables`, the `load_audit` table, the model `evaluation_summary`, and a sample of `consumption_daily`.

4.3 Interactive dashboard (live Streamlit)

GreenPower Utilities — Energy Analytics Dashboard

Group 5 · IIT Jodhpur · built on the Week 4 feature layer

Avg daily consumption

26.2 kWh

Peak-to-average ratio

1.92

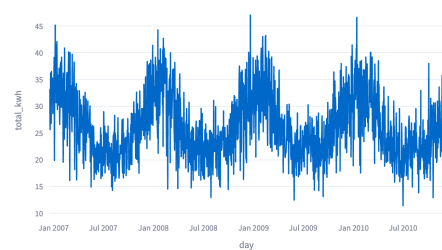
Mean daily peak

2.07 kW

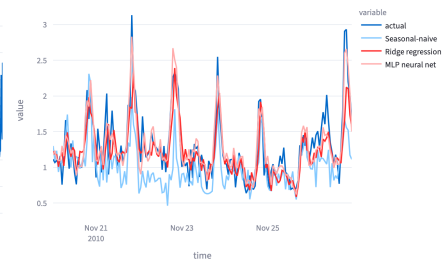
Median peak hour

20:00

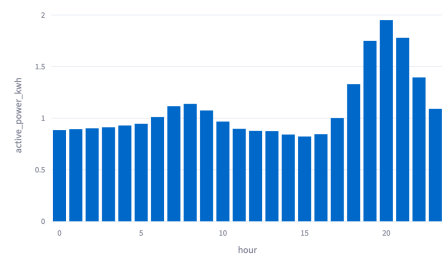
Daily consumption trend



Forecast vs actual (final week)



Load profile by hour



Model evaluation

	model	MAE_kw	RMSE_kw	MAPE_pct
0	Seasonal-naive	0.299	0.392	28.5
1	Ridge regression	0.179	0.24	17.3
2	MLP neural net	0.136	0.177	13.9

Figure 3 — The live Streamlit dashboard at localhost:8501, reading the same SQLite database and pipeline outputs: KPIs, daily-consumption trend, forecast-vs-actual, hourly load profile and the model-evaluation table.

4.4 Analytical figures

The six report figures below are rendered by `src/viz.py` directly from the live feature tables, forecasts and anomaly flags.

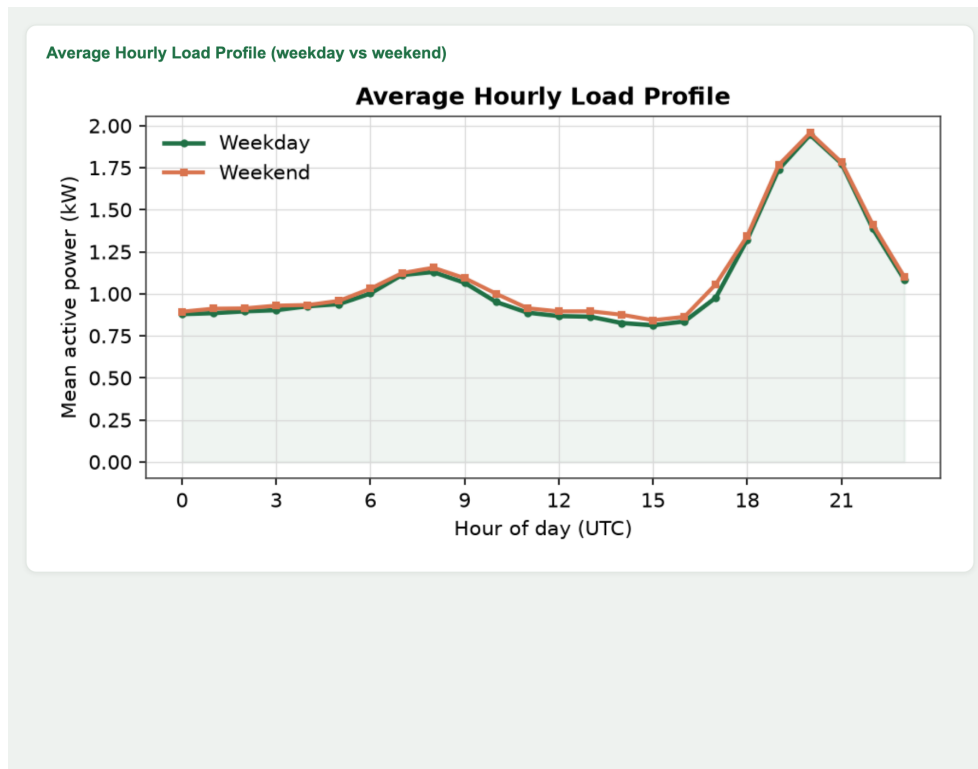


Figure 4 — Average hourly load profile; a pronounced ~20:00 evening peak, slightly flatter on weekends.

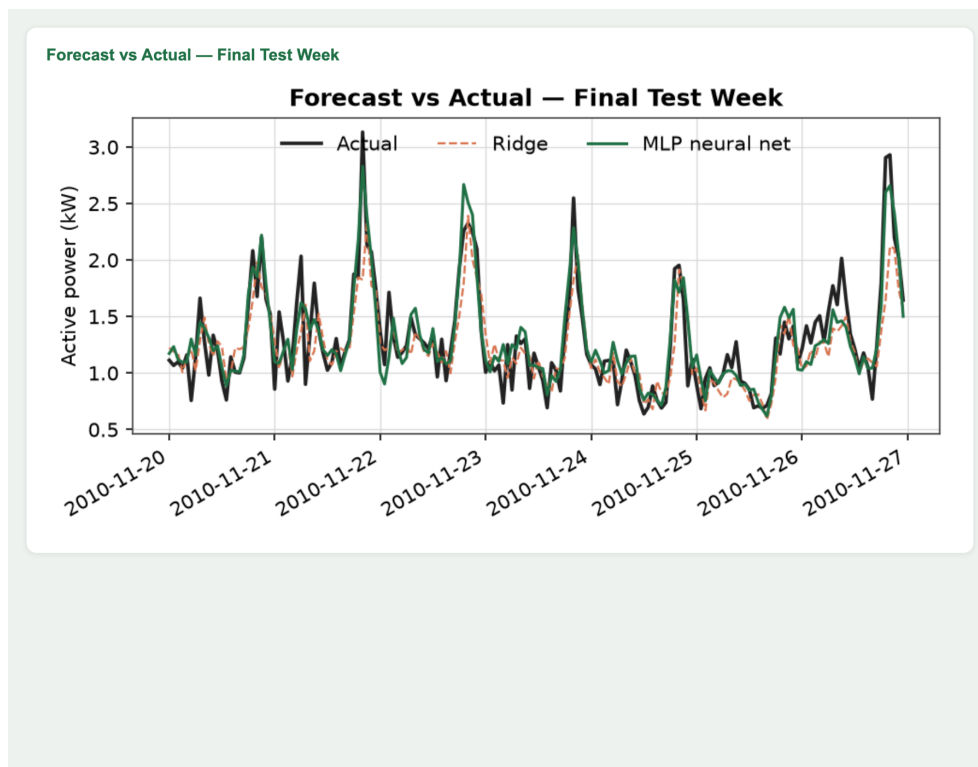


Figure 5 — Forecast vs actual over the final test week; the MLP tracks the actual load closely.

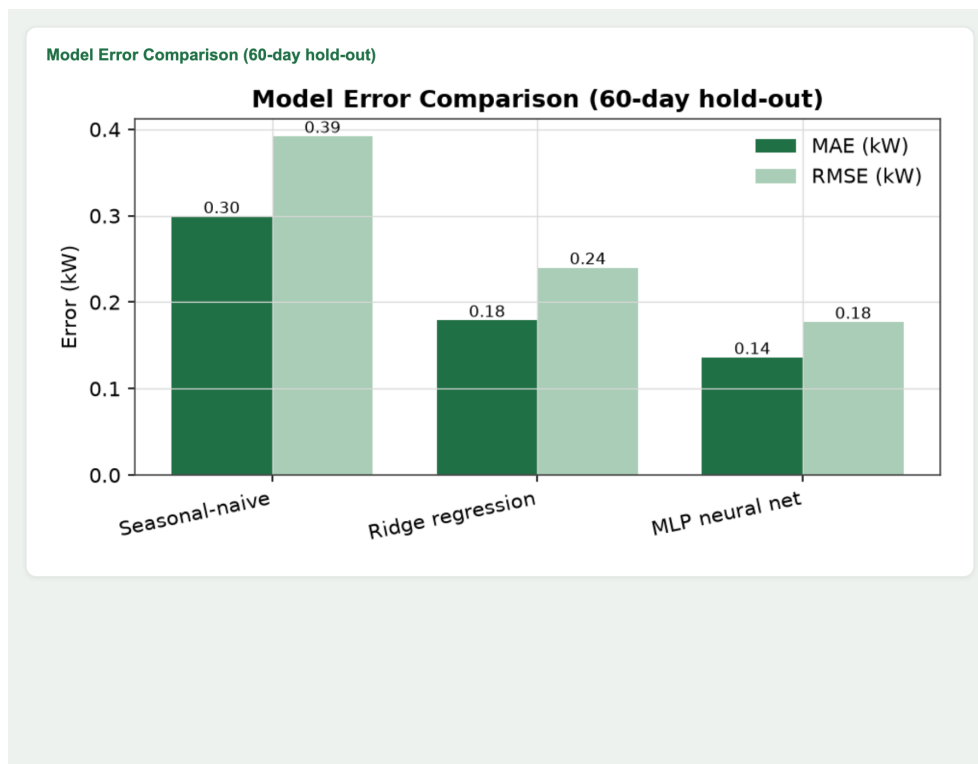


Figure 6 — MAE / RMSE across the three models on the 60-day hold-out; error falls steadily from baseline to MLP.

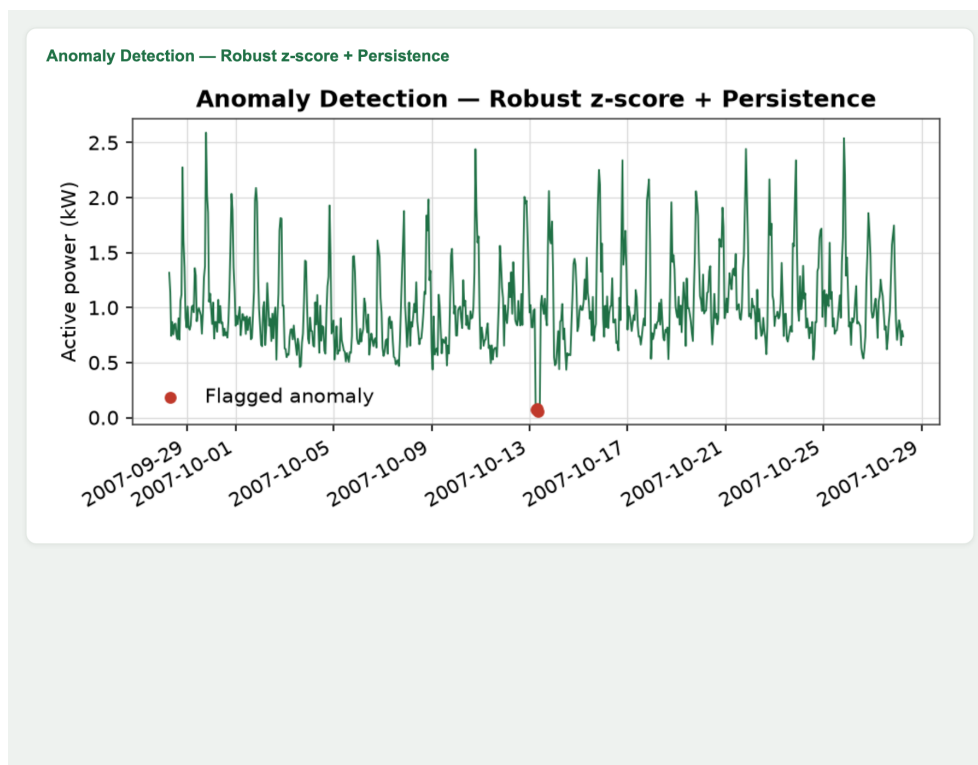


Figure 7 — Anomaly detector flagging injected outages and spikes via robust z-score + persistence.

Daily Consumption vs Temperature

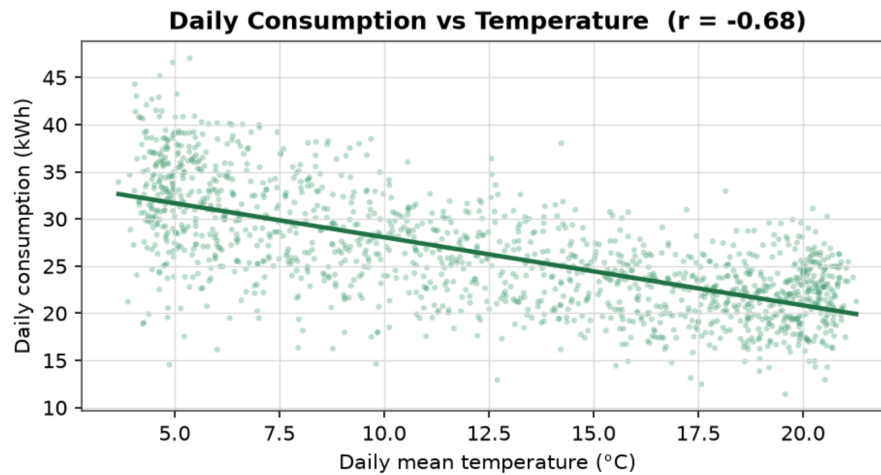


Figure 8 — Daily consumption vs temperature; clear negative (heating-driven) relationship.

Wind Capacity Factor by Month

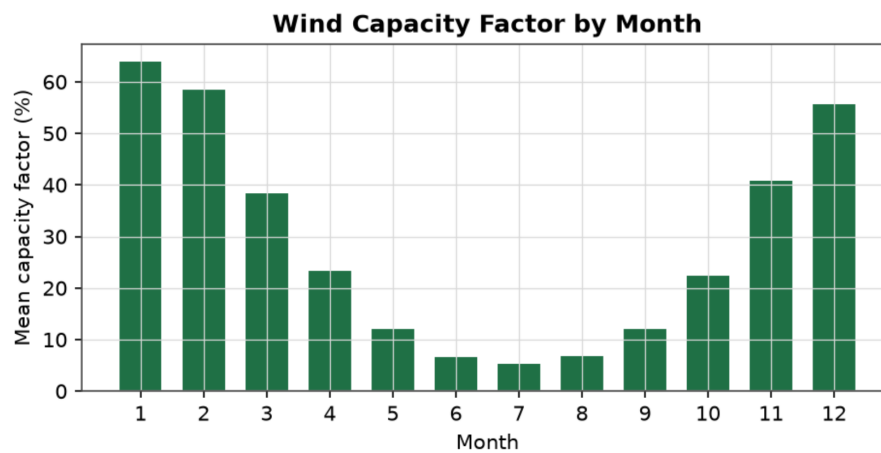


Figure 9 — Wind capacity factor by month; higher in the windier winter months.

5. Results

5.1 Cleaning statistics (Week 2)

Dataset	Raw rows	Clean rows	Spikes removed	Gaps imputed	Completeness
Consumption	34,505	34,608	27	268	99.61%
Generation	21,823	21,888	10	162	99.63%
Weather	123,120	123,120	0	0	100.00%

5.2 Feature-layer statistics (Week 4)

Feature statistic	Value
Hourly feature rows	34,608
Daily / monthly rollup rows	1,442 / 48
Generation daily rows	912
Peak-to-average ratio (mean)	1.92
Load factor (mean)	0.529
Median peak hour	20:00
Evening-peak share (17–21h)	99.0%
r(load, temp) hourly / daily	-0.362 / -0.677
r(load, heating degree-hours) daily	0.689
Wind capacity factor (mean)	29.9%

5.3 Forecasting model evaluation (Week 5)

60-day chronological hold-out — 1,440 test hours, 33,000 train hours, split at 2010-09-27.

Model	MAE (kW)	RMSE (kW)	MAPE (%)
Seasonal-naive	0.299	0.392	28.5
Ridge regression	0.179	0.24	17.3
MLP neural net	0.136	0.177	13.9

Best model: MLP neural net — MAPE 13.9%, RMSE 0.177 kW, a 55% RMSE reduction versus the seasonal-naive baseline.

5.4 Anomaly detection (Week 5)

Metric	Value
Injected anomaly hours	42
Flagged hours	38

True positives	37
False positives	1
False negatives	5
Precision	0.97
Recall	0.88

6. Model Methodology

Forecasting. The target is the household's next-hour active power. Three models of increasing sophistication are compared on an identical 60-day chronological hold-out, using lag features (1h, 24h, 168h), 24-hour rolling mean/std, cyclical hour/day-of-year encodings, a weekend flag and heating/cooling degree-hours:

Model	Role	Implementation
Seasonal-naive	baseline	value 168 h earlier
Ridge regression	linear	scikit-learn, $\alpha = 1.0$
MLP neural net	non-linear (runnable)	scikit-learn, hidden (64, 32)
LSTM (Keras)	production deep model	TensorFlow, 168-step lookback

When TensorFlow is installed the pipeline additionally trains the Keras LSTM (`models.train_lstm_keras`); otherwise the MLP fills the neural, non-linear role — as in this run. Error is reported as MAE, RMSE and a clipped MAPE (denominator floored at 0.3 kW to avoid divide-by-near-zero blow-up at low overnight load).

Anomaly detection. Each reading is compared to the median for its hour-of-day $\times$ month group, and the deviation is scaled by a per-group median absolute deviation so naturally variable evening hours are not over-flagged. A two-part rule fires on either a sustained deviation (≥ 2 consecutive hours, characteristic of an outage) or a single hard spike. Evaluation injects a labelled set of synthetic outages and spikes and reports precision / recall.

Storage. The runnable engine is SQLite with the same logical schema as the production TimescaleDB design; loads are idempotent and audited (see `load_audit`). The production migrations add hypertables, one-month chunking, native compression and continuous aggregates.

7. Code → Six-Week Mapping

Week	Deliverable	Code / Output
1	Data sourcing & profiling	<code>data_acquisition.py</code> ; calibrated to Week 1–2 statistics
2	Cleaning & preprocessing	<code>clean.py</code> → <code>cleaning_report.json</code> (despike, impute, align)
3	Time-series storage	<code>storage.py</code> (SQLite) + <code>db/migrations/*.sql</code> (TimescaleDB)
4	Feature engineering	<code>features.py</code> → <code>feature_stats.json</code> , feature tables/views
5	Forecasting & anomaly detection	<code>models.py</code> + <code>anomaly.py</code> → <code>evaluation_summary.csv</code> , stats JSON
6	Visualization, dashboard & report	<code>viz.py</code> , <code>dashboard/*</code> , this report

All figures and metrics in this report were generated directly by the pipeline run documented in Section 4 — no values were transcribed by hand.